SYNOPSES

Project:- Advanced Web-Based Electronic Circuit Simulation

GLA University

Greater Noida Campus

Name:- Aawahan Bansal

Univ. Roll no:- 2251500004

Project Synopsis: Interactive Web-Native Electronic Circuit Simulator

1. Project Overview and Rationale

This project details the architecture and implementation of a high-performance, interactive electronic circuit simulator running natively within the browser environment. By utilizing HTML5 Canvas, CSS3, and Vanilla JavaScript, the system provides a platform-independent, dependency-free CAD environment for real-time circuit analysis and visualization.

1.1 Architectural Shift: Legacy Java to Modern JavaScript

The primary engineering objective was the migration from legacy plugin-based systems (specifically Java Applets) to a modern JavaScript-based execution model. To preserve the numerical stability of the foundational logic developed by Paul Falstad, the original Java source was cross-compiled using the Google Web Toolkit (GWT). This architectural bridge ensures that the technical core of the simulation remains almost untouched, inheriting decades of refined solver logic while leveraging the performance of contemporary JIT-compiled JavaScript interpreters.

1.2 Target Audience and Utility

The simulator serves as a high-fidelity visualization tool for:

- **Electronic Engineering Students:** Bridging the gap between nodal equations and physical behavior through animated transients.
- **Hobbyists:** Providing a rapid, low-friction prototyping environment.
- **Professional Designers:** Offering a lightweight tool for sub-circuit verification and Driving Point Impedance (DPI) exploration without the overhead of heavy SPICE suites.

2. Mathematical and Numerical Foundations

2.1 Equation Formulation: MNA and DPI Integration

The simulation engine utilizes **Modified Nodal Analysis (MNA)** to formulate the system's global Jacobian. While standard Nodal Analysis is limited to admittance-based elements, MNA allows for the direct inclusion of independent voltage sources and current-dependent elements by introducing branch currents as additional variables.Crucially, the solver incorporates **Driving Point Impedance (DPI) analysis** . While MNA provides the global system structure, the DPI approach facilitates the solution of individual node impedances, effectively modularizing the interaction between the linear solver and the component-level state updates.

2.2 Numerical Discretization and Element Stamps

The system populates the MNA matrix by "stamping" the contributions of each component. For reactive elements, the simulator employs **Companion Models** , where capacitors and inductors are modeled as a parallel combination of a conductance and a companion current source in the discretized time domain.**Element Stamps (Backward Euler Integration Focus):**| Element Type | Matrix Location (i, j) | Contribution (Admittance) | RHS Contribution (Current) | Notes || ------ | ------ | ------ | ------ | ------ || **Resistor (R)** | $(i, i), (j,

j)$ | $+G$ | 0 | $G = 1/R$ (Admittance formulation) || | $(i, j), (j, i)$ | $-G$ | 0 | || **Voltage Source (E)** | $(i, k), (k, i)$ | $+1$ | V_{src} | Adds branch current k as a variable || | $(j, k), (k, j)$ | -1 | | Grounded terminal required || **Capacitor (C)** | $(i, i), (j, j)$ | $+C/h$ | $I_{eq} = (C/h) \cdot V_{prev}$ | h = time step; modeled via BE || | $(i, j), (j, i)$ | $-C/h$ | $-I_{eq}$ | RHS represents past state energy |

2.3 Linear System Solvers and Pivoting Strategy

The engine employs Direct Methods for solving linear algebraic equations, specifically **LU Factorization** via Gaussian Elimination. To ensure the matrix remains numerically well-behaved and to minimize fill-ins, the solver follows the **Ho et al. pivoting strategy** :

1. **Singleton Processing:** Rows/columns with a single ± 1 entry (typically from grounded voltage sources or current sources) are processed first to avoid fill-ins.
2. **Row Interchange:** For branch relations of voltage sources (E) and inductors (L), rows are interchanged with nodal equations to maintain a strong diagonal and prevent numerical cancellation.
3. **Sparsity Optimization:** The remaining diagonal elements are selected as pivots to maintain numerical stability.

2.4 Nonlinear Analysis and Integration

Nonlinear networks (diodes, BJTs, MOSFETs) are solved via the **Newton-Raphson** iteration scheme. Elements are linearized at each step into companion models (conductance in parallel with a current source).For dynamic circuits, the simulator supports Forward Euler (FE), Backward Euler (BE), and the Trapezoidal Rule (TR). To mitigate the "Trapezoidal Ringing" inherent in TR during stiff transitions, the engine can employ **smoothing techniques** or transiently revert to Backward Euler to ensure absolute stability.

3. Technical Architecture and Data Flow

3.1 Software Module Hierarchy

The application follows a strict data-to-solver pipeline:

- **Netlist.js (Data):** Acts as the primary wrapper, managing the circuit’s structural state.
- **Spice_asc.js (Parser):** Interprets standard SPICE netlists and .asc files, converting topological definitions into a machine-readable format.
- **Circuit.js (Solver):** The core computational engine. It manages the global Jacobian, executes the DPI analysis, and iterates the MNA solver loop.

3.2 Technological Constraints

- **Topological Constraints:** The current implementation does not support "super-node" analysis; consequently, all voltage sources must be grounded on one side (no floating voltage sources).
- **Component Scope:** Optimized for independent/controlled sources, R , C , and L .
- **Dependency-Free:** Pure Vanilla JS and HTML5 Canvas ensure zero external overhead.

4. Functional Features and User Interface

4.1 Schematic Capture and Diagnostics

The interface supports a real-time "Select/Drag" workflow for drawing and resizing components.

- **Diagnostic Overlays:** If the simulator detects unconnected nodes that prevent matrix convergence, it renders a **red circle** at the terminal to identify the topological break.

4.2 Real-time Animated Visualization

- **Color-Coded Gradients:** Green indicates positive voltage, red indicates negative, and grey represents ground.
- **Current Vectoring:** Moving yellow dots visualize current magnitude and direction.
- **Interactive Probing:** Mouse-over actions display the instantaneous state (voltage, current, power) in the info area.

4.3 Virtual Oscilloscopes

Integrated scope views provide concurrent graphing of voltage (green) and current (yellow). Scopes include automatic peak detection and mouse-over highlighting, allowing users to trace graph lines back to specific physical components in the schematic.

4.4 Parameter Interactivity

Component values can be modulated during active simulation via UI sliders (ideal for transducers) or the mouse wheel, which cycles through standard **E12 range** values for resistors, capacitors, and inductors.

5. Performance and Operational Parameters

5.1 Simulation vs. Real Life

The simulator assumes ideal models to maximize computational speed:

- **Zero-Resistance Leads:** Wires and connectors have no parasitic resistance.
- **Gate Input Characteristics:** Logic gate inputs draw zero current, effectively modeling ideal CMOS behavior (contrasted with 1980s-era TTL which had significant input bias).
- **Infinite Source Capacity:** Voltage sources will attempt to provide infinite current unless constrained by series resistance.

5.2 Error Handling

1. **Singular Matrix:** Occurs from inconsistent nodes (e.g., shorted voltage sources) or undefined potentials.
2. **Voltage Source/Capacitor Loops:** Illegal loops with no series resistance, resulting in infinite current states.
3. **Convergence Failed:** Newton-Raphson failure, typically requiring a simulation reset to escape local minima in nonlinear models.

6. Strategic Development Roadmap

- **Phase 1: Matrix Efficiency:** Implementation of **Sparsity-aware storage** formats (CSR/CSC) and integration of the **Markowitz Criteria** for pivot selection to handle larger, more complex networks with minimal fill-in.
- **Phase 2: Hierarchical Support:** Development of true sub-circuit support for modular design and multi-node scope analysis.
- **Phase 3: Extended Transducer Library:** Modeling of thermistors and light-dependent resistors (LDRs) using electrically equivalent variable models controlled via the existing slider API.
- **Phase 4: High-Frequency Modes:** Expanding analysis to RF and fast digital transients by reducing default time steps from 5µs to the picosecond range (e.g., the **5ps step** used in transmission line examples).

7. Licensing and Contributions

The project is governed by the **GNU General Public License (GPLv2)** . It acknowledges the seminal work of **Paul Falstad** and the cross-compilation efforts that enabled the transition of this legacy Java logic into a high-performance, web-native JavaScript simulator. All source code is maintained through open-source contribution models on GitHub.